D2 Debugging tools

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track D: Finding errors in code 2
D2 Debugging tools 3
D2-L1 breakpoint() and pdb, stop and look 3
D2-L2 Logging levels that help at 3 a.m. 7
D2-L3 explain(), the Spark UI and the query profile 12
Track D best-practice checklist 18
Appendix: solutions and answer keys 19

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D2 Debugging tools

Roadmap: Every month. breakpoint(), pdb, logging levels, Spark UI and explain().

D2-L1 breakpoint() and pdb, stop and look

Concept

A traceback shows where code failed. A **debugger** shows the values at that moment. Python ships one: **pdb**.

Stop at a line. Put `breakpoint()` in your code. When Python reaches it, pdb opens a prompt, `(Pdb)`, with every local variable in scope. Setting the environment variable `PYTHONBREAKPOINT=0` turns every `breakpoint()` call into a no-op, so a forgotten one cannot hang a scheduled job. Remove them before you commit anyway.

Stop where it crashed. Run a script under pdb with post-mortem on:

```
python -m pdb -c continue run_qa.py
...
ZeroDivisionError: float division by zero
Uncaught exception. Entering post mortem debugging
> /repo/run_qa.py(2)weighted_pct()
-> ratio = reach / base
(Pdb) p reach, base
(5.0, 0.0)
```

You land in the frame that raised, with its variables intact. Checked on Python 3.13.

The commands you use most:

Command	Does
`p expr`, `pp expr`	Print, or pretty-print, a value
`w`	Show the stack, newest frame last
`u`, `d`	Move up to the caller, or down again
`n`	Run the next line, stepping over calls
`s`	Step into a call
`c`	Continue to the next breakpoint
`l`	List source around the current line
`q`	Quit

A useful pattern: after a crash, `p` the inputs of the failing frame. If they look wrong, `u` to the caller and `p` what it passed. Repeat until you find the first frame where a value went bad. That is the frame to fix.

In Databricks notebooks, the built-in interactive debugger gives breakpoints, stepping and a variable view for Python cells. It runs on serverless and on recent runtimes, after you turn it on in your editor settings.

In Spark code a debugger stops only the driver. Code inside a UDF runs on executors, where your breakpoint never fires. Test UDF logic as a plain Python function first.

Why it matters

A weighted-reach function returned 0.0 for one subgroup. An analyst added eleven print statements over two hours and still missed it. A post-mortem pdb session on a forced failure showed in one command that `base` was the unweighted count, passed in by a caller that swapped two arguments. `u`, then `p`, found the caller.

Worked example: weak code

```
1  def weighted_pct(reach: float, base: float) -> float:
2      """Weighted percentage."""
(1)    print("DEBUG reach", reach)
4      print("DEBUG base", base)
(2)    breakpoint()
6      return round(reach / base * 100, 1)
```

1. **Line 3.** Print debugging leaves noise in job logs and shows only what you guessed to print.
2. **Line 5.** Committed by mistake, it waits for input in a job that nobody is watching.

Worked example: strong code

```
1  def weighted_pct(reach: float, base: float) -> float | None:
2      """Weighted percentage, or None when the base is zero."""
(1)    if base == 0:
4          return None
5      return round(reach / base * 100, 1)
6
(2) # To investigate a failure, run: python -m pdb -c continue run_qa.py
8  # then: p reach, base   u   p rows
```

1. **Line 3.** The fix the debugging session pointed to: guard the division.
2. **Line 7.** Debug from outside the code. Nothing to remove before you commit.

Let the crash open the debugger, print the failing frame's inputs, then walk up until the values go wrong.

Recap card

- **Open pdb where a script crashed** python -m pdb -c continue script.py. You land in the frame that raised.
- **pdb: print, stack, up, next, step, continue** p, w, u, n, s, c.
- **PYTHONBREAKPOINT=0** Makes breakpoint() a no-op. A safety net for jobs, not a reason to commit breakpoints.
- **Why a breakpoint in a Spark UDF never fires** UDFs run on executors. The debugger stops only the driver. Test UDF logic as plain Python.

Practice

Exercise 1. Multiple choice (level 1)

A scheduled job runs with `PYTHONBREAKPOINT=0`. The code contains `breakpoint()`. What happens at that line?

- a) Nothing: the call does nothing and the job continues
- b) The job waits for input forever
- c) The job fails with a RuntimeError
- d) pdb opens and quits at once

Why?

- r1) PYTHONBREAKPOINT=0 makes breakpoint() a no-op.
- r2) Scheduled jobs never run pdb.

Exercise 2. Fill the gap (level 2)

You are in a post-mortem pdb session in `weighted_pct`. Complete the commands.

```
(Pdb) _____(1) reach, base        # print the inputs
(Pdb) _____(2)                    # move to the caller's frame
(Pdb) p rows
```

Exercise 3. Put in order (level 2)

Order a debugging session for a function that returns a wrong number.

- Stop in the function with breakpoint() or post-mortem pdb
- Print the inputs, walk up with u until a value is first wrong
- Reproduce with the smallest input that gives the wrong number
- Fix that frame and add a test with the small input

- Remove the breakpoint and run the tests

Exercise 4. Write the function (level 3)

Post-mortem debugging reads the local variables of the frame that raised. Write `crash_locals(fn: Callable[..., object], *args: object) -> dict[str, object] | None` that calls `fn(*args)`. If it raises, return a copy of the local variables of the **innermost** frame of the traceback (the one pdb lands in). If it does not raise, return `None`. Import `Callable` from `collections.abc`.

What you need to know.

An exception carries its traceback in `err.__traceback__`. A traceback is a chain of frames, outermost first: `tb.tb_next` moves one call deeper and is `None` at the innermost frame. `tb.tb_frame` is the frame itself, and `tb.tb_frame.f_locals` holds its local variables as a dict.

```
def inner():
    raise ValueError

def outer():
    count = 3
    inner()

try:
    outer()
except Exception as err:
    tb = err.__traceback__              # the frame that called outer()
    tb.tb_next.tb_frame.f_locals        # {'count': 3}: outer's locals
```

Your code must pass these checks:

- Return the locals of the innermost frame, where the error was raised, not the caller's or your own.
- Return None when the call succeeds.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Local variables of the frame that raised, as pdb post-mortem shows them.
def crash_locals(
    fn: Callable[..., object],
    *args: object,
) -> dict[str, object] | None: ...
```

Starter code:

```
def crash_locals(fn, *args):
    try:
        fn(*args)
    except Exception:
        return locals()
```

Exercise 5. Spot the bug (level 3)

This function is ready for a scheduled job, except for one line. Tap it, then remove it and anything else left from debugging.

```
1 def unweighted_base(rows: list[dict[str, object]]) -> int:
2     """Count in-scope respondents."""
3     in_scope = [r for r in rows if r["in_scope"] is True]
4     breakpoint()
5     return len(in_scope)
```

Write down the line number of the bug. Then write the fix.

Sources

- python -m pdb -c continue enters post-mortem at the raising frame and prints 'Uncaught exception. Entering post mortem debugging'; PYTHONBREAKPOINT=0 disables breakpoint(); commands p, w, u, d, n, s, c, l, q. (verified: Local runs on CPython 3.13.12, September 2026.)
- Databricks notebooks have a built-in interactive debugger for Python, available on serverless and recent runtimes, turned on in editor settings. (verified: https://docs.databricks.com/aws/en/notebooks/debugger (checked 2026-09-25))
- ruff rule T100 flags debugger calls such as breakpoint(). (verified: ruff 0.16.9 run in this repo reports 'T100 Trace found: `breakpoint` used'.)

D2-L2 Logging levels that help at 3 a.m.

Concept

A job that failed overnight leaves you its logs and nothing else. Good logs answer three questions fast: what ran, with what inputs, and where it stopped. Python's `logging` module does this better than `print`.

One logger per module:

```
1 import logging
2
3 logger = logging.getLogger(__name__)
```

Levels, lowest to highest, with their numbers:

Level	Number	Use it for
DEBUG	10	Detail you need only while investigating
INFO	20	Normal progress: step started, rows written
WARNING	30	Unexpected but handled: a WARN check

Level	Number	Use it for
ERROR	40	A step failed
CRITICAL	50	The pipeline cannot continue

A logger passes on records at or above its effective level. The root logger's default is **WARNING**, so `logger.info(...)` prints nothing until you configure logging. Checked on Python 3.13:

- `logging.basicConfig(level=logging.INFO)` configures the root logger **once**. A second call does nothing, unless you pass `force=True`.
- `logger.exception("...")` inside an `except` block logs at ERROR and appends the traceback.
- Pass values as arguments: `logger.info("wrote %d rows to %s", n, table)`. The string is built only if the record is emitted. ruff rule G004 flags f-strings in logging calls.

Logging is not control flow. `logger.critical(...)` records a message. It does not stop anything. Your standard says a CRITICAL check **raises**. Log, then raise.

Logging is not your run log. Text logs are for people debugging. The Delta run log table, with one structured row per run, is for queries and dashboards. Write both.

Every log line in a pipeline should carry the identifiers that let you find the data: tracker, wave, table, run id.

Why it matters

A silver job logged "Processing..." and "Done" at INFO for every wave, and nothing else. When wave W14 produced 30% fewer rows, the logs could not say which step dropped them. Adding one INFO line per step, with the row count, tracker and wave, turned the next investigation from a day of reruns into a two-minute log search.

Worked example: weak code

```
import logging

def run_checks(results: list[dict[str, str]], wave_id: str) -> None:
    """Report check results."""
    for r in results:
        if r["status"] == "FAIL" and r["severity"] == "CRITICAL":
            logging.critical(f"check failed: {r['check']}")
```

```
(2)          print("checked", r["check"])
```

1. **Line 7.** Logs CRITICAL and carries on. The pipeline continues with bad data. The f-string is built even when filtered (G004).
2. **Line 8.** print has no level, no logger name and no wave id.

Worked example: strong code

```
 1 import logging
 2
(1) logger = logging.getLogger(__name__)
 4
 5
 6 class CriticalCheckFailed(RuntimeError):
 7     """A CRITICAL data check failed and the pipeline must stop."""
 8
 9
10 def run_checks(results: list[dict[str, str]], wave_id: str) -> None:
11     """Log each check result and stop on a CRITICAL failure."""
12     for r in results:
(2)          logger.info("wave %s check %s status %s", wave_id, r["check"],
   r["status"])
14         if r["status"] == "FAIL" and r["severity"] == "CRITICAL":
15             logger.critical("wave %s CRITICAL check %s failed", wave_id,
   r["check"])
(3)             raise CriticalCheckFailed(f"{r['check']} failed for wave
   {wave_id}")
```

1. **Line 3.** A named logger per module. The name shows up in every record.
2. **Line 13.** Identifiers in every line, formatted only when emitted.
3. **Line 16.** Log, then raise. The level describes. The exception stops.

One named logger per module, levels that match severity, identifiers in every line, and a raise after every CRITICAL.

Recap card

- **Logging levels and numbers** DEBUG 10, INFO 20, WARNING 30, ERROR 40, CRITICAL 50. Default effective level: WARNING.
- **basicConfig called twice** The second call does nothing, unless force=True.
- **logger.exception** Logs at ERROR with the traceback. Call it inside an except block.
- **Does logger.critical stop the pipeline?** No. Log, then raise.

Practice

Exercise 1. Predict the output (level 1)

Nothing has configured logging. What does this print?

```
import logging

logger = logging.getLogger("qa")
logger.info("rows written")
logger.warning("weight sum low")
```

Exercise 2. Multiple choice (level 2)

A notebook calls `logging.basicConfig(level=logging.WARNING)` in its first cell, then `logging.basicConfig(level=logging.DEBUG)` later. What is the root level after the second call?

- a) WARNING: the second call does nothing
- b) DEBUG
- c) An error is raised
- d) Both handlers are active

Why?

- r1) basicConfig does nothing if the root logger already has handlers, unless force=True.
- r2) Logging levels can never be lowered.

Exercise 3. Fill the gap (level 2)

Log the failure with its traceback, then stop the pipeline.

```
try:
    validate(df)
except ContractError:
    logger._____(1)("contract failed for wave %s", wave_id)
    _____(2)
```

Exercise 4. Write the function (level 3)

Write `log_check(logger: logging.Logger, check: str, status: str, severity: str, wave_id: str) -> None`. Log at INFO for `"PASS"`, WARNING for `"WARN"`, ERROR for `"FAIL"` with non-CRITICAL severity, and CRITICAL for `"FAIL"` with `"CRITICAL"` severity, then raise `RuntimeError`. Every message contains the wave id and check name. Use %-style arguments, not f-strings, in the logging calls. Raise `ValueError` for an unknown status.

What you need to know.

`logging.Logger` is the type of the object `logging.getLogger(name)` returns. Each level is a number: `logging.INFO` is 20, `WARNING` 30, `ERROR` 40, `CRITICAL` 50. `logger.log(level, message, *args)` logs at a level you pass in, so a dict can map each status to its level.

Pass values as arguments after a `%s` template. Logging fills them in only when the message is actually emitted:

```
logger.log(logging.WARNING, "wave %s has %d rows", "W1", 0)
# WARNING wave W1 has 0 rows
```

Your code must pass these checks:

- PASS is INFO, WARN is WARNING, a non-critical FAIL is ERROR. Include the wave and check in each message.
- A CRITICAL failure logs at CRITICAL and then raises RuntimeError.
- Unknown statuses raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
STATUS_LEVELS = ...

# Log one check result at a level matching its outcome.
def log_check(
    logger: logging.Logger,
    check: str,
    status: str,
    severity: str,
    wave_id: str,
) -> None: ...
```

Starter code:

```
import logging


def log_check(logger, check, status, severity, wave_id):
    logger.info(f"{check} {status}")
```

Exercise 5. Architecture design (level 4)

Set the logging for a silver job that runs nightly on Databricks.

Options: INFO, with DEBUG switched on by a job parameter when investigating, DEBUG always.

1. Level in production
2. Each step's log line
3. Run history for dashboards

Sources

- Default effective level WARNING; unconfigured output is the bare message; basicConfig second call no-op unless force=True; logger.exception logs at ERROR with traceback; level numbers 10 to 50. (verified: Local run on CPython 3.13.12, September 2026.)
- ruff rule G004 flags f-strings in logging calls. (verified: ruff 0.16.9 run in this repo reports 'G004 Logging statement uses f-string'.)

D2-L3 explain(), the Spark UI and the query profile

Concept

A slow Spark job often shows why in its **plan**. Read it before you tune anything.

`df.explain()` prints the physical plan without running the query. `df.explain("formatted")` numbers each operator and lists its details below. The other modes are `"simple"` (the default), `"extended"`, `"codegen"` and `"cost"`. Read the tree from the **bottom up**: the scan is at the bottom, the result at the top.

What to look for (examples from PySpark 4.0.4):

In the plan	Means
`PartitionFilters: [(wave_id = 3)]`	Only matching partitions are read. Good.
`PushedFilters: [GreaterThan(w, 1.0)]`	The filter runs inside the file reader. Good.
`PushedFilters: []` on a filtered query	Nothing was pushed down. Check the filter's column and type.
`Exchange hashpartitioning(...)`	A shuffle: data moves across the cluster. Each one costs time.
`BroadcastHashJoin`	The small side is copied to every executor. No shuffle of the large side.
`SortMergeJoin`	Both sides are shuffled and sorted. Normal for two large tables.
`CartesianProduct` or `BroadcastNestedLoopJoin`	Every row meets every row. Usually a missing join condition.
`BatchEvalPython`	A Python UDF. Rows leave the JVM for a Python worker.

`AdaptiveSparkPlan isFinalPlan=false` means adaptive query execution will adjust the plan at run time, for example switching a join to broadcast. `explain()` shows the starting plan.

After the run, look at what happened. On classic compute, the **Spark UI** shows jobs, stages, tasks and the SQL tab with row counts per operator. On **serverless**, the Spark UI is not available. Use the **query profile**, which shows each operator with its time spent, rows and memory peak, and a Top operators panel.

In the UI or profile, compare **rows out** of each join with rows in. A join whose output exceeds its larger input is duplicating rows (lesson D3-L1). Look for one task far slower than the rest of its stage: that is skew.

Why it matters

A gold job's runtime grew from 4 minutes to 70 over a month. Nobody had changed the code. `explain()` showed `BroadcastNestedLoopJoin`: a join on a range condition, fine while the dimension held 50 rows, painful at 4,000. The plan named the problem in one line. Rewriting the range join as an equi-join on a precomputed band key brought it back to 5 minutes.

Worked example: weak code

```
1 from pyspark.sql import functions as F
2
3 joined = respondents.crossJoin(regions).filter(
4     F.col("postcode_area") == F.col("area_code")
5 )
6 joined.write.mode("overwrite").saveAsTable("research_gold.nat_regions")
```

1. **Line 3.** A cross join first, then a filter. Check the plan before you trust the optimiser to rewrite it.
2. **Line 6.** Runs without ever reading the plan.

Worked example: strong code

```
1 from pyspark.sql import functions as F
2
3 joined = respondents.join(
4     F.broadcast(regions), respondents["postcode_area"] ==
    regions["area_code"], "left"
5 )
6 joined.explain("formatted")
7 # Expect BroadcastHashJoin, no CartesianProduct, one scan of respondents.
8 joined.write.mode("overwrite").saveAsTable("research_gold.nat_regions")
```

1. **Line 4.** An explicit equi-join with the small side broadcast.

2. **Line 6.** Read the plan once, when you write the code, and again when data volumes change.

Read the plan bottom up, look for pushed filters, shuffles, join types and Python UDFs, then confirm rows per operator in the Spark UI or query profile.

Recap card

- **Read a physical plan in which direction?** Bottom up: scan at the bottom, result at the top.
- **PartitionFilters and PushedFilters** Filters applied while reading. Empty PushedFilters on a filtered query means no pushdown.
- **Plan operators that signal trouble** CartesianProduct or BroadcastNestedLoopJoin (cross join), BatchEvalPython (Python UDF), many Exchange (shuffles).
- **Spark UI on serverless** Not available. Use the query profile: time spent, rows and memory per operator.

Practice

Exercise 1. Multiple choice (level 1)

A job's plan contains `CartesianProduct`. What is the most likely cause?

- a) A join without an equality condition, so every row meets every row
- b) A broadcast join
- c) A Python UDF
- d) Partition pruning

Why?

- r1) A Cartesian product pairs every row with every row. It comes from a cross join or a join condition Spark cannot use as an equi-join.
- r2) Spark uses Cartesian products for all small tables.

Exercise 2. Spot the bug (level 2)

This is the formatted plan for `spark.read.parquet(path).filter(F.col("w") > 1.0).groupBy("wave_id").agg(F.sum("w"))`. Tap the line that shows a shuffle.

```
1 AdaptiveSparkPlan (6)
2 +- HashAggregate (5)
3    +- Exchange (4)
4       +- HashAggregate (3)
5          +- Filter (2)
6             +- Scan parquet  (1)
```

Write down the line number of the bug.

Exercise 3. Multiple choice (level 2)

Your job runs on **serverless** compute. Where do you see time spent and rows per operator after it runs?

- a) The query profile
- b) The Spark UI's SQL tab
- c) df.explain('cost')
- d) The driver logs

Why?

- r1) The Spark UI is not available on serverless. Databricks points you to the query profile.
- r2) The query profile is the only tool that works in Databricks.

Exercise 4. Write the function (level 3)

Write `plan_warnings(plan: str, max_exchanges: int = 2) -> list[str]` that reads the text of `df.explain()` and returns warnings, in this order, each at most once: `"cross join"` if the plan contains `CartesianProduct` or `BroadcastNestedLoopJoin`, `"python udf"` if it contains `BatchEvalPython` or `ArrowEvalPython`, and `"many shuffles"` if the word `Exchange` appears more than `max_exchanges` times, not counting `BroadcastExchange`.

What you need to know.

`re.findall(pattern, text)`, or `compiled.findall(text)`, returns every match as a list, so `len(...)` counts them. Two pattern tools control what counts:

- `\b` marks a word boundary: `\bcat\b` matches `cat` as a whole word only.
- `(?<!con)` is a **negative lookbehind**: the match counts only when `con` does not come right before it.

```python
import re

text = "cat concat bobcat"
re.findall(r"cat", text)           # ['cat', 'cat', 'cat']
re.findall(r"(?<!con)cat", text)   # ['cat', 'cat']: not the one in concat
re.findall(r"\bcat\b", text)       # ['cat']
```

Your code must pass these checks:

- BroadcastNestedLoopJoin is a cross join too, and BatchEvalPython is a Python UDF. Keep the order.
- BroadcastExchange is not a shuffle. Count only plain Exchange operators.
- Three shuffles is more than the default of 2.
- No warnings for a normal plan.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
CROSS_JOIN_OPERATORS = ...

# Module constant.
PYTHON_UDF_OPERATORS = ...

# Module constant.
SHUFFLE_PATTERN = ...

# Warnings for common problems in a Spark physical plan.
def plan_warnings(
    plan: str,
    max_exchanges: int = 2,
) -> list[str]: ...
```

Starter code:

```
def plan_warnings(plan, max_exchanges=2):
    warnings = []
    if "Cartesian" in plan:
        warnings.append("cross join")
    return warnings
```

Exercise 5. Architecture design (level 4)

A gold job on classic compute slowed from 5 to 40 minutes. Plan the investigation.

Options: Run explain('formatted') and read the joins and exchanges, Double the cluster size.

1. First step
2. Checking what happened at run time
3. One task in a stage takes 30 minutes, the rest 20 seconds

Sources

- explain modes simple, extended, codegen, cost, formatted; plans show PartitionFilters, PushedFilters, Exchange hashpartitioning, BroadcastHashJoin, SortMergeJoin, CartesianProduct, BatchEvalPython, AdaptiveSparkPlan isFinalPlan=false. (verified: Local runs on PySpark 4.0.4, September 2026.)
- The Spark UI is not available on serverless; use the query profile, which shows time spent, rows and memory per operator and a Top operators panel. (verified: https://docs.databricks.com/aws/en/compute/serverless/

limitations and https://docs.databricks.com/aws/en/sql/user/queries/query-profile (checked 2026-09-25))

Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

D2-L1 breakpoint() and pdb, stop and look

D2-L1 exercise 1

Hints. (1) The variable controls what breakpoint() calls. (2) 0 means disabled. (3) The job continues.

Answer. a) Nothing: the call does nothing and the job continues Because: r1) PYTHONBREAKPOINT=0 makes breakpoint() a no-op.

Set PYTHONBREAKPOINT=0 in job environments as a safety net, and still remove breakpoints before committing.

Common wrong answers.

- `b`: That is what happens without the variable. With 0, the call is ignored.
- `c`: No error. The call returns immediately.
- `d`: pdb never starts.
- `a|r2`: Right answer, wrong reason. Without the variable, a scheduled job would stop at the breakpoint.

D2-L1 exercise 2

Hints. (1) Print is one letter. (2) Up is one letter. (3) `p` and `u`.

Answer.

```
(Pdb) p reach, base        # print the inputs
(Pdb) u                    # move to the caller's frame
(Pdb) p rows
```

`p` shows values. `u` moves to the caller, where the bad value came from.

Common wrong answers.

- g2 = `d`: `d` moves down, towards the newest frame. You are already there. Use `u`.

D2-L1 exercise 3

Hints. (1) What makes each run fast? (2) Find the first wrong value. (3) Clean up last.

Answer.

1. Reproduce with the smallest input that gives the wrong number
2. Stop in the function with breakpoint() or post-mortem pdb
3. Print the inputs, walk up with u until a value is first wrong
4. Fix that frame and add a test with the small input
5. Remove the breakpoint and run the tests

Small reproduction, stop, walk up to the first wrong value, fix with a test, clean up.

Common wrong answers.

- Reproduce first, with small data, so each run is fast and you know the right answer.
- Fix and add the test while you know the cause, then clean up.

D2-L1 exercise 4

Hints. (1) `err.__traceback__` is the outermost traceback entry. (2) Follow `tb_next` until it is None. (3) `tb.tb_frame.f_locals` holds that frame's variables.

Answer.

```python
from collections.abc import Callable


def crash_locals(fn: Callable[..., object], *args: object) -> dict[str, object] | None:
    """Local variables of the frame that raised, as pdb post-mortem shows them.

    Args:
        fn: Function to call.
        *args: Arguments for fn.

    Returns:
        A copy of the innermost frame's locals, or None if fn did not raise.
    """
    try:
        fn(*args)
    except Exception as err:
        tb = err.__traceback__
        while tb is not None and tb.tb_next is not None:
            tb = tb.tb_next
        return dict(tb.tb_frame.f_locals) if tb is not None else None
    return None
```

A traceback is a linked list from your call down to the raise. pdb post-mortem opens at its last entry.

D2-L1 exercise 5

Hints. (1) Which line would stop an unattended job? (2) A debugging call. (3) Line 4.

Answer. The bug is on line 4.

Fixed version:

```
1 def unweighted_base(rows: list[dict[str, object]]) -> int:
2     """Count in-scope respondents."""
3     in_scope = [r for r in rows if r["in_scope"] is True]
4     return len(in_scope)
```

A leftover `breakpoint()` blocks a job waiting for input that never comes. ruff rule T100 flags it.

Common wrong answers.

- Line 3: `is True` is correct for a Python bool flag in a dict. The problem is the next line.

D2-L2 Logging levels that help at 3 a.m.

D2-L2 exercise 1

Hints. (1) What is the root logger's default level? (2) WARNING. (3) Only the warning appears, as bare text.

Answer.

```
1 weight sum low
```

With no configuration, records at WARNING and above go to stderr as the bare message. INFO is dropped.

Common wrong answers.

- `rows written weight sum low`: The default effective level is WARNING, so the INFO record is dropped.
- `WARNING:qa:weight sum low`: That is basicConfig's default format. Without configuration, Python's last-resort handler prints the message only.

D2-L2 exercise 2

Hints. (1) Does basicConfig reconfigure? (2) Only the first call counts. (3) force=True changes that.

Answer. a) WARNING: the second call does nothing Because: r1) basicConfig does nothing if the root logger already has handlers, unless force=True.

Pass `force=True`, or set levels on named loggers with `setLevel`.

Common wrong answers.

- `b`: Only with force=True, or with logger.setLevel.
- `c`: It returns silently, which is why this surprises people.
- `d`: No second handler is added.
- `a|r2`: Right answer, wrong reason. setLevel or force=True lower the level fine.

D2-L2 exercise 3

Hints. (1) Which logger method adds the traceback? (2) A bare statement re-raises the current exception. (3) `exception` and `raise`.

Answer.

```python
try:
    validate(df)
except ContractError:
    logger.exception("contract failed for wave %s", wave_id)
    raise
```

`logger.exception` logs at ERROR with the traceback. A bare `raise` re-raises the same exception.

Common wrong answers.

- g1 = `error`: error logs the message without the traceback. exception adds it.
- g2 = `pass`: Swallowing the error lets the pipeline continue with invalid data.

D2-L2 exercise 4

Hints. (1) A dict from status to level keeps the branches short. (2) `logger.log(level, msg, *args)` logs at any level. (3) Handle the CRITICAL case first: log, then raise.

Answer.

```python
import logging

STATUS_LEVELS = {"PASS": logging.INFO, "WARN": logging.WARNING, "FAIL":
    logging.ERROR}


def log_check(
```

```
    logger: logging.Logger, check: str, status: str, severity: str,
    wave_id: str
) -> None:
    """Log one check result at a level matching its outcome.

    Args:
        logger: Module logger.
        check: Check name.
        status: "PASS", "WARN" or "FAIL".
        severity: Severity of the check, such as "CRITICAL".
        wave_id: Wave being checked.

    Raises:
        ValueError: If status is unknown.
        RuntimeError: If a CRITICAL check failed.
    """
    if status not in STATUS_LEVELS:
        raise ValueError(f"unknown status: {status}")
    if status == "FAIL" and severity == "CRITICAL":
        logger.critical("wave %s check %s failed (CRITICAL)", wave_id,
    check)
        raise RuntimeError(f"CRITICAL check {check} failed for wave
    {wave_id}")
    logger.log(STATUS_LEVELS[status], "wave %s check %s %s", wave_id,
    check, status)
```

Levels describe. Exceptions stop. A CRITICAL failure needs both.

D2-L2 exercise 5

Hints. (1) What do you read at 3 a.m.? (2) Which identifiers find the data? (3) Where does history for dashboards live?

Answer.

- INFO keeps logs readable. A parameter enables DEBUG without a code change.
- Identifiers and counts let you find the step that dropped rows.
- Structured history belongs in a table. Text logs are for people.

INFO by default, identifiers everywhere, and a structured run log alongside the text.

Common wrong answers.

- level = `debug`: DEBUG every night buries the lines you need under detail you rarely read.
- content = `done`: 'Done' cannot tell you which wave or how many rows.
- record = `parse`: Log text changes. A run log table with a fixed schema is queryable.

D2-L3 explain(), the Spark UI and the query profile

D2-L3 exercise 1

Hints. (1) What does Cartesian mean? (2) Every pair of rows. (3) Check the join condition.

Answer. a) A join without an equality condition, so every row meets every row Because: r1) A Cartesian product pairs every row with every row. It comes from a cross join or a join condition Spark cannot use as an equi-join.

Find the join that produced it and give it an equality condition on the keys.

Common wrong answers.

- `b`: Broadcast joins show as BroadcastHashJoin.
- `c`: Python UDFs show as BatchEvalPython or ArrowEvalPython.
- `d`: Pruning shows as PartitionFilters on the scan.
- `a|r2`: Right answer, wrong reason. Small tables get BroadcastHashJoin when there is an equality condition.

D2-L3 exercise 2

Hints. (1) Which operator moves data across the cluster? (2) Its name means swap. (3) Exchange.

Answer. The bug is on line 3.

`Exchange hashpartitioning(wave_id, 200)` shuffles partial sums so each wave's rows meet on one executor for the final sum.

Common wrong answers.

- Line 6: The scan reads files. The shuffle moves data between executors, which is an Exchange.
- Line 2: HashAggregate aggregates. The final aggregate needs the data shuffled first, by the Exchange below it.

D2-L3 exercise 3

Hints. (1) Is the Spark UI available on serverless? (2) No. (3) Its replacement.

Answer. a) The query profile Because: r1) The Spark UI is not available on serverless. Databricks points you to the query profile.

On serverless, use the query profile. On classic compute, the Spark UI's SQL tab shows the same kind of detail.

Common wrong answers.

- `b`: Correct for classic compute. Serverless has no Spark UI.

- `c`: explain shows the plan before running, not what happened.
- `d`: Logs do not show per-operator metrics.
- `a|r2`: Right answer, wrong reason. Classic compute has the Spark UI too.

D2-L3 exercise 4

Hints. (1) Keep the operator names in constants. (2) A negative lookbehind, `(?<!Broadcast)`, excludes BroadcastExchange. (3) Append warnings in the stated order.

Answer.

```python
import re

CROSS_JOIN_OPERATORS = ("CartesianProduct", "BroadcastNestedLoopJoin")
PYTHON_UDF_OPERATORS = ("BatchEvalPython", "ArrowEvalPython")
SHUFFLE_PATTERN = re.compile(r"(?<!Broadcast)\bExchange\b")


def plan_warnings(plan: str, max_exchanges: int = 2) -> list[str]:
    """Warnings for common problems in a Spark physical plan.

    Args:
        plan: Text of df.explain().
        max_exchanges: Number of shuffles allowed before warning.

    Returns:
        Warnings in a fixed order, each at most once.
    """
    warnings: list[str] = []
    if any(op in plan for op in CROSS_JOIN_OPERATORS):
        warnings.append("cross join")
    if any(op in plan for op in PYTHON_UDF_OPERATORS):
        warnings.append("python udf")
    if len(SHUFFLE_PATTERN.findall(plan)) > max_exchanges:
        warnings.append("many shuffles")
    return warnings
```

A plan linter in a test catches a cross join the day it appears, not the day the dimension grows.

D2-L3 exercise 5

Hints. (1) What is free to check? (2) Where are rows and task times? (3) What does one slow task mean?

Answer.

- The plan shows join types and shuffles for free, before you spend compute.
- Rows per operator show where rows multiply. Task times show skew.

- One very slow task among fast ones is the signature of a skewed key.

Plan first, then runtime metrics, then the data behind any outlier.

Common wrong answers.

- first = `size`: A bigger cluster makes a cross join or a skewed key slightly less slow, at double the cost.
- runtime = `rerun`: A rerun tells you it is still slow, not why.
- skew = `slow_node`: Skew follows the data, not the machine. It returns after a restart.